Arduino Microcontroller Concepts

Chapter 5A • Simple Digital and Analog Input

DIGITAL INPUT

Program: B. TECH., 7th Semester

SIKSHA 'O' ANUSHANDHAN (Deemed to be) UNIVERSITY

Learning goals

- Distinguish digital input states HIGH and LOW and configure Arduino input pins.
- Understand switch interfaces and why an input must have a defined logic state.
- Use pull-up and pull-down arrangements, including INPUT_PULLUP.
- Recognize switch bounce and determine press duration using timing.
- Understand row–column scanning in a matrix keypad.

Digital input: the basic idea

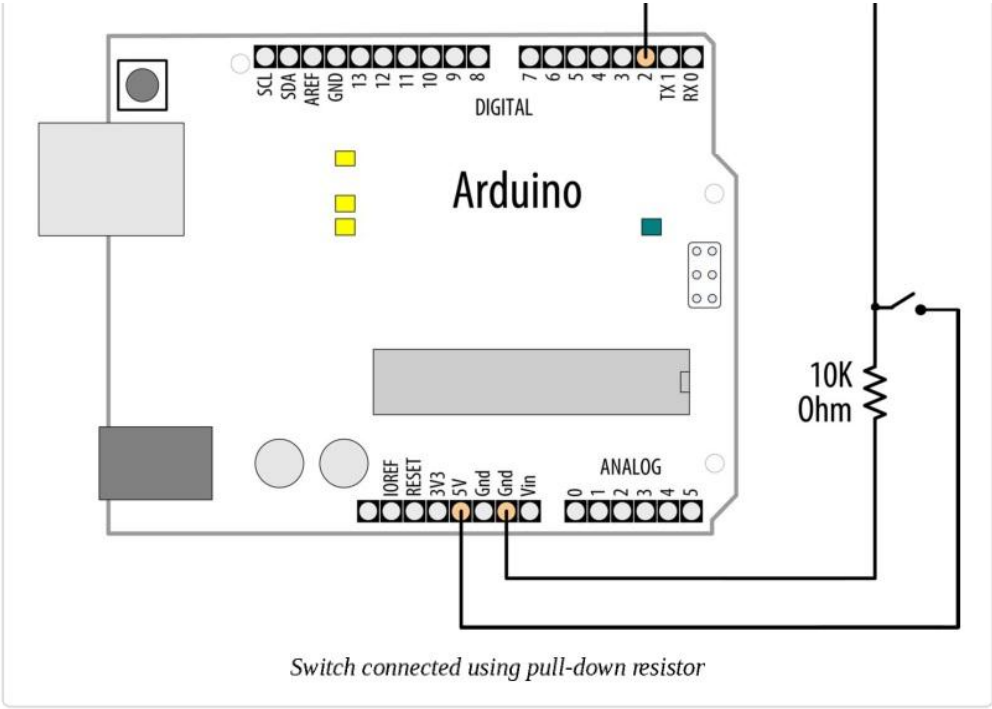
- A digital input is interpreted as one of two logic states: HIGH or LOW.
- digitalRead(pin) returns the present logic state of a digital input.
- pinMode(pin, INPUT) configures a pin as an input.
- A push-button, switch, or digital sensor may act as a digital source.
- The important engineering question: what state does the pin see when the device is inactive?

What is a switch electrically?

- A switch is a controlled electrical connection: open means the path is broken; closed means the path conducts.
- Normally-open (NO): the contact is open until the switch is actuated.
- Normally-closed (NC): the contact is closed until the switch is actuated.
- In a microcontroller circuit, the switch itself does not automatically create a valid HIGH or LOW level.
- A resistor network is often used to define the inactive state.

Floating input: why it is a problem

- When the switch is open, an input without a bias path can float.
- A floating CMOS input can pick up electrical noise and produce unpredictable HIGH/LOW transitions.
- The software may appear to “see” presses that never happened.
- Provide a definite pull-up or pull-down path to force a known state.



Switch connected using pull-down resistor

Textbook circuit: switch with pull-down resistor

Pull-up vs pull-down

- Pull-down: resistor holds the input LOW when the switch is open; pressing the switch can connect the input to the positive supply.
- Pull-up: resistor holds the input HIGH when the switch is open; pressing the switch can connect the input to ground.
- Both methods prevent a floating input.
- Choose the arrangement that makes the circuit logic and code simplest.

Using the Arduino internal pull-up


```
pinMode(7, INPUT_PULLUP);
switchState = digitalRead(7);
```

Switch open

Internal pull-up holds the input HIGH.

Switch pressed

Button connects the input to ground → LOW.

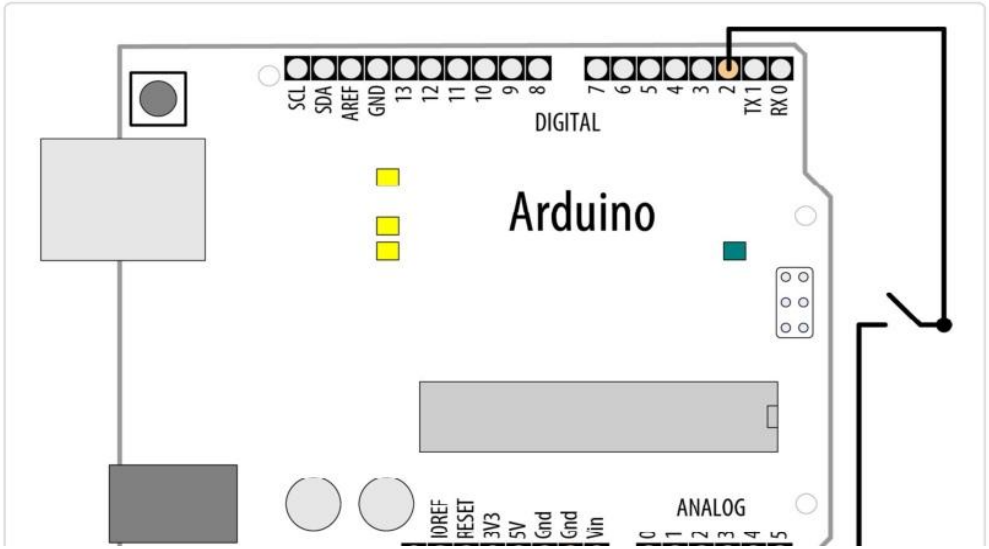
Exam point

With INPUT_PULLUP: released → HIGH, pressed → LOW (active-LOW logic).

Solution

As explained in Recipe 5.1, digital inputs must have a resistor to hold the pin to a known value when the switch is not pressed. Arduino has internal pull-up resistors that can be enabled by using the `INPUT_PULLUP` mode with `pinMode`.

For this example, the switch is wired as shown in Figure 5-5. This is almost exactly the same as Figure 5-4, but without an external resistor.



Textbook circuit: INPUT_PULLUP arrangement

pinMode() and digitalRead()

- pinMode(pin, mode) configures a pin as INPUT, OUTPUT, or INPUT_PULLUP.
- digitalRead(pin) reads HIGH or LOW from a digital input.
- Configure the pin once in setup(); read the pin repeatedly in loop().
- For a switch, always interpret the electrical arrangement before writing the if-condition.

A complete switch → LED example


```
const int button = 7;
const int LED = 13;

void setup() {
  pinMode(button, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}

void loop() {
  if (digitalRead(button) == LOW)
    digitalWrite(LED, HIGH);
  else
    digitalWrite(LED, LOW);
}
```

Read the hardware first

INPUT_PULLUP makes the pressed state LOW.

Then write the logic

LOW → LED ON; HIGH → LED OFF.

Programming pattern

configure → read → interpret → act

Switch bounce

- A mechanical contact does not always change from one state to the other cleanly.
- During a real press, the electrical signal can oscillate rapidly before settling.
- One physical press may therefore look like several digital transitions.
- Debouncing converts the noisy transition into one reliable logical event.

Debouncing: practical logic

Problem

rapid transitions

→

Decision

is the state stable?

→

Action

accept the new state only after stability

```
if (digitalRead(2) == LOW) {
    delay(50);                // debounce
    if (digitalRead(2) == LOW) {
        count++;
    }
}
```

Measuring switch-press duration

Press duration = release time − press time

Example

Pressed = 1.25 s

Example

Released = 2.87 s

Result

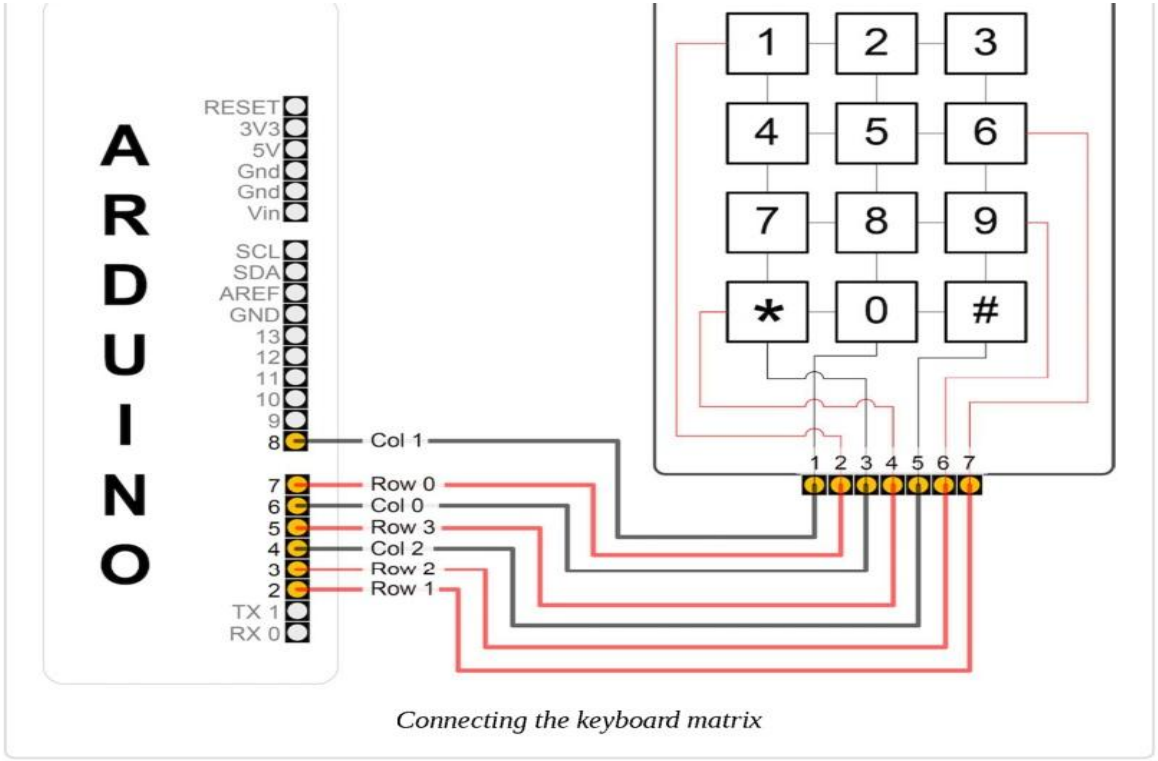
Duration = 1.62 s

Implementation idea: record the start time when the press begins; subtract it from the current time when the switch is released.

Why timing matters in CSE applications

- A push-button may represent a user command, menu selection, timer start, or alarm acknowledge signal.
- Press duration can distinguish a short press from a long press.
- Debouncing protects the higher-level application logic from the electrical behavior of the switch.
- This is the bridge from a physical component to event-driven software.

Matrix keypad: more than one switch



Connecting the keyboard matrix

If you've wired your Arduino and keypad as shown in Figure 5-6, the following sketch will print key presses to the Serial Monitor:

```
/*
  Keypad sketch
```

Textbook wiring example

Rows

4 rows in the 4 × 3 keypad

Columns

3 columns in the 4 × 3 keypad

Signal lines

4 + 3 = 7 lines

Keypad as a matrix of switches

- A 4 × 3 keypad physically contains 12 key positions.
- Instead of one Arduino pin per key, the keys are organized into rows and columns.
- Only 7 signal lines are needed for a 4 × 3 matrix.
- Scanning identifies the row and column associated with the pressed key.
- This is a useful example of hardware organization reducing I/O requirements.

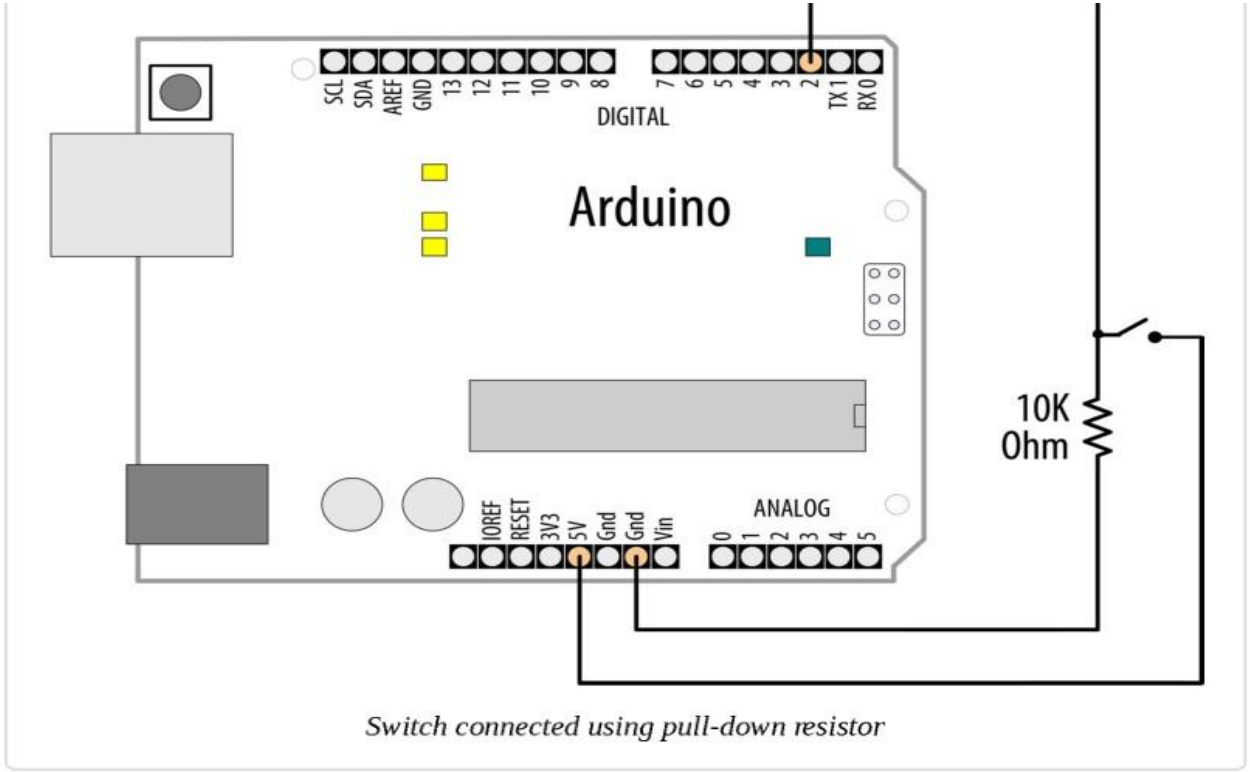
How row–column scanning works

- 1. Drive or activate one row/column group.
- 2. Read the other group of lines.
- 3. Detect which line changed state.
- 4. The active row + active column identify the pressed key.
- Libraries automate the scanning details, but the electrical idea is row–column addressing.

The resistor behind the logic

- A pull-up or pull-down resistor provides a weak bias so an inactive input settles to a known state.
- The resistor should not waste unnecessary current, yet must be strong enough to reject ordinary noise.
- The switch overrides the bias when actuated.
- This is a simple example of using a passive component to condition a signal before software reads it.

Pull-down connection: read the circuit, not just the code



Switch connected using pull-down resistor

Switch open

Resistor pulls the input LOW.

Switch closed

Switch connects the input to the supply → HIGH.

Logic consequence

The code must treat HIGH as the pressed state in this arrangement.

Textbook switch with pull-down resistor

Pull-up vs pull-down: a quick comparison

Arrangement	Inactive state	Pressed state	Typical code logic
Pull-down	LOW	HIGH	if (digitalRead(pin) == HIGH)
Pull-up	HIGH	LOW	if (digitalRead(pin) == LOW)

Same hardware idea, different active logic. Always derive the code from the circuit.

Debouncing on a timeline

Digital input



Debounce logic waits for a stable interval before accepting the transition as a genuine button event.

Why it matters

One physical press should create one software event.

Timing tool

millis() lets the program compare elapsed time without blocking the whole application.

2-Mark Question • Digital input

Exam-oriented review

State the logic levels obtained from a push-button connected to Arduino digital pin 7 using the internal pull-up resistor. Also identify the pin-configuration statement required for this interface.

Key points

- Open switch → HIGH; pressed switch → LOW.
- pinMode(7, INPUT_PULLUP);
- Pressing the switch connects the input to ground.

CO / HHH mapping

CO2 • L3 (Head)

Write the answer compactly but include all requested elements.

2-Mark Question • Debouncing + timing

Exam-oriented review

Consider a mechanical push-button that shows multiple rapid HIGH/LOW transitions during one press. Determine the purpose of debouncing and calculate the press duration if the detected press and release times are 1.25 s and 2.87 s.

Key points

- Debouncing removes false multiple transitions.
- Duration = 2.87 − 1.25 = 1.62 s.
- Know both the electrical issue and the timing calculation.

CO / HHH mapping

CO2 • L3 (Head)

Write the answer compactly but include all requested elements.